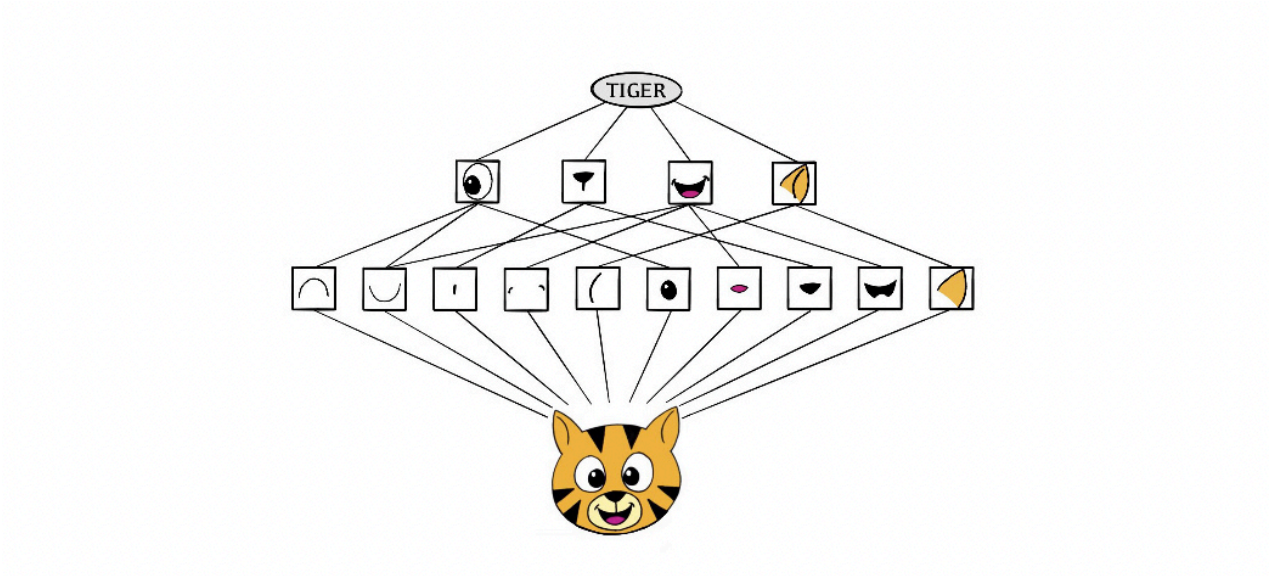
CNNs = Convolutional Neural Networks

Geek4Geek and ISLP

It is popular around 2010.
CNNs also known as **ConvNets** are widely used for Computer vision tasks.

- It mimics to some degree to how human classify images, by recognizing **specific features or patterns anywhere in the images that distinguish each particular object class.**



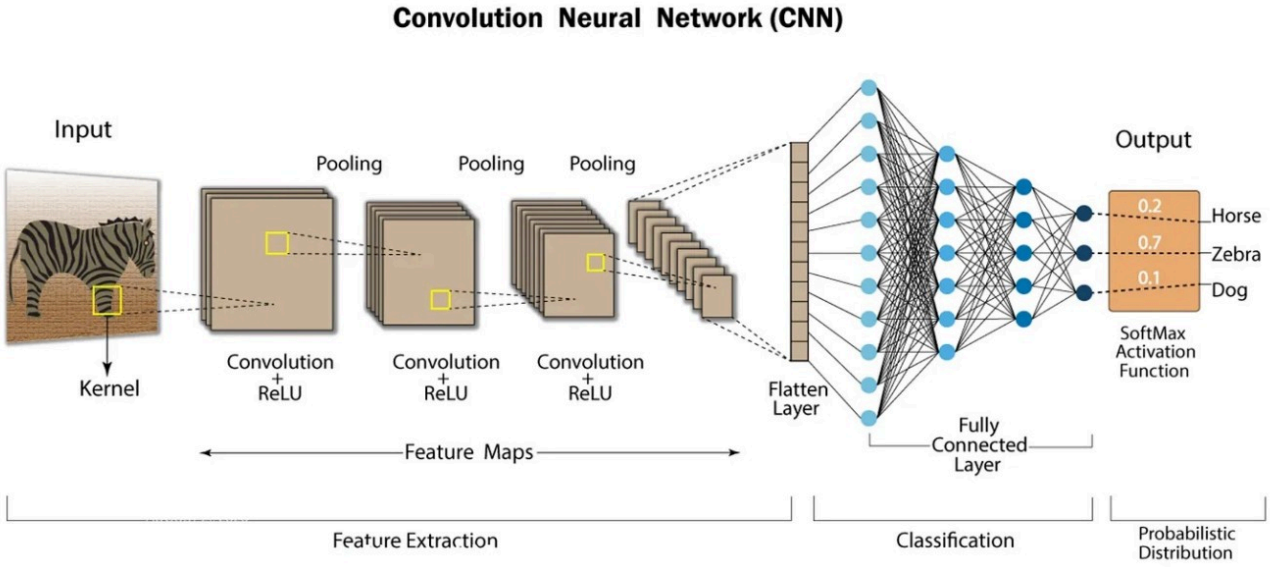
  The network first identifies low-level feature in the input image, such as small edge, patches of color, and the like. then the low-features are combined to form higher-level features, such as part the ears, mouth and so on. eventually the higher-level feature contribute to the probability of any given output class.

It combines two specialized type of hidden layers, called **convolutional layers** and **pooling layers**.

The convolutional layer search for instances of small patterns in the image, whereas **pooling layers** downsample these to select prominent subset.

- Automatically learn hierarchical features through convolution operations, from simple edges and textures to complex shapes and objects.
- Detect objects at different positions within an image, ensuring robustness to spatial variations.
- Reduce computational complexity by processing local regions instead of the entire image at once.

Overview of CNNs

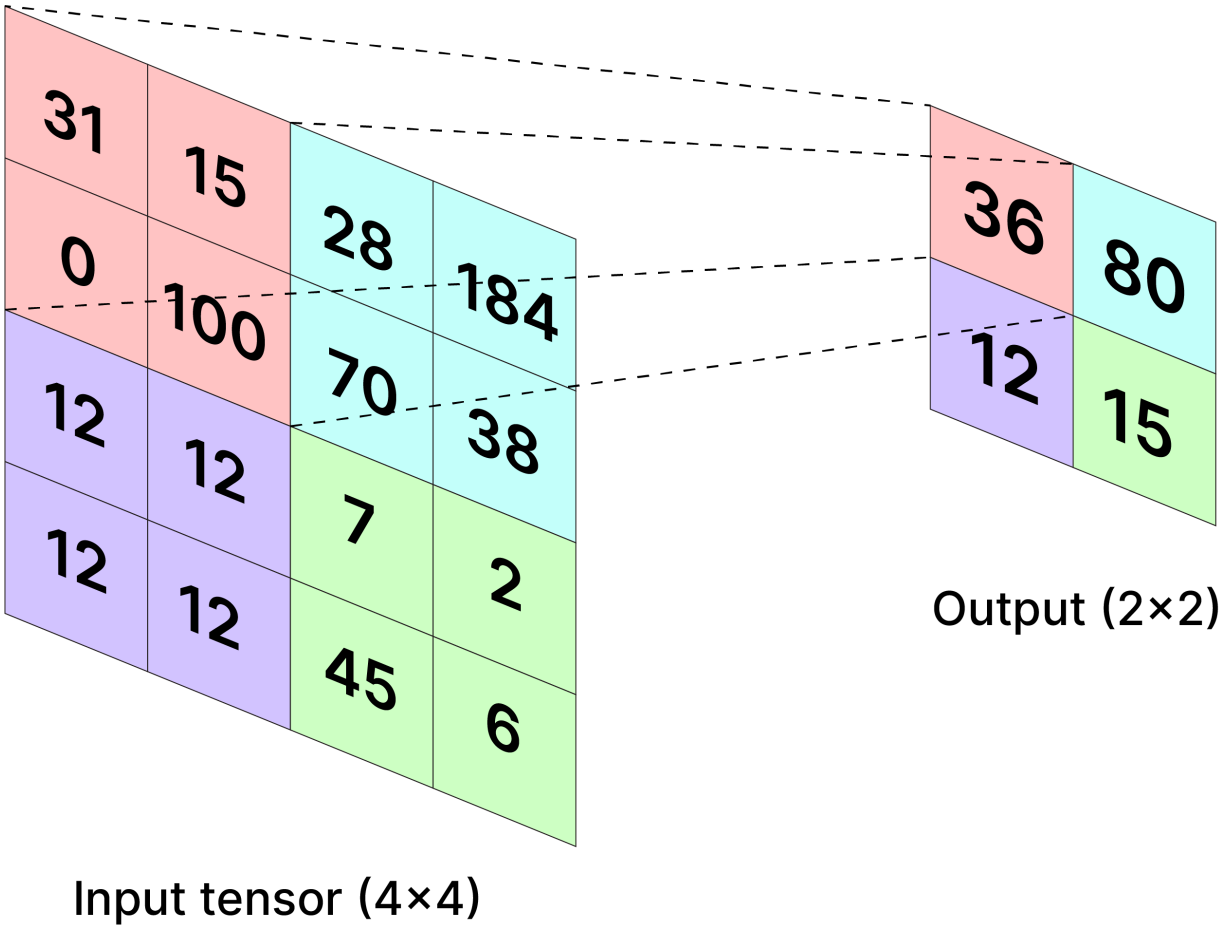


1. **Input Layer**
   It receives the raw image data and passes it to the network for processing. Typically, the input is a 3D volume (Width x height x depth)
   - Stores pixel values of the image (e.g., 32 × 32 × 3 for RGB images).
   - Preserves the spatial structure of the image for further feature extraction.
2. **Convolutional Layer**
   It is responsible for extracting important features from the input data. It applies a set of **learnable filters (kernels)** that slide over the image and compute the dot product between the filter weights and corresponding image patches, producing **feature maps**.
   - Uses small filters (e.g., 2x2, 3x3, 5x5) to scan the input image.
   - Generates feature maps that capture patterns such as edges, textures and shapes.
     Example: Using 12 filters results in an output volume of 32 × 32 × 12.
3. **Activation Layer**
   It introduces non-linearity into the network by applying an **element-wise activation function** to the output of the convolutional layer. This enables the model to learn complex patterns beyond linear relationship.
   - Common activation functions include ReLU, Tanh and Leaky ReLU.
   - Applied element-wise to the feature maps.
   - The output dimensions remain unchanged (e.g., 32 × 32 × 12).
4. **Pooling Layer**
   It condense the **large image** into a **smaller summary image**
   It is used to reduce the spatial dimensions of the feature maps, which help:

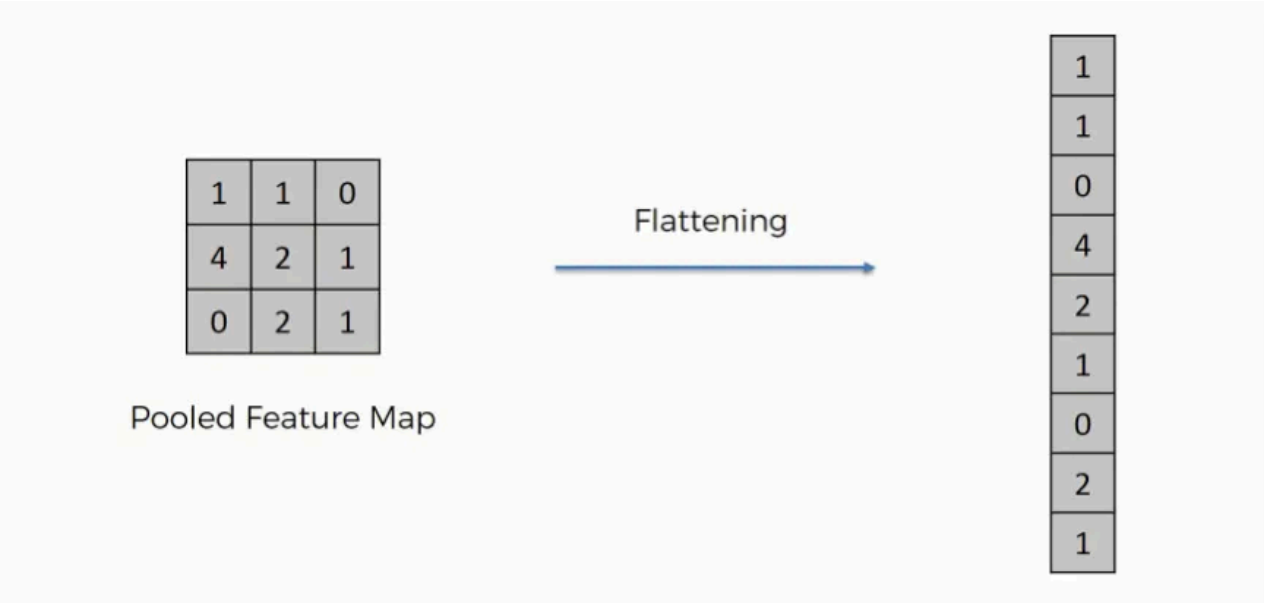
   - making computation faster,
   - reducing memory usage and
   - **preventing overfitting**.

   it is typically inserted between convolutional layers in a CNN.
   - Common types include **Max Pooling** and **Average Pooling**.

- Reduces **width** and **height** while keeping depth unchanged.
  Example: Using 2 x 2 max pooling with stride 2 reduces the volume from 32 x 32 x 12 to 16 x 16 x 12.



5. **Flattening**
   It converts the **multi-dimensional feature maps** into **a 1-dimensional array for feeding next layer after convolution and pooling**. this vector is then passed to the fully connected layer for classification or regression.
   Example: Flattening 16 x 16 x 12 results in a vector of size 3072 (16 x 16 x 12)



6. **Fully Connected Layer = Connected Components**
   The fully connected (dense) layer performs high-level reasoning using extracted features

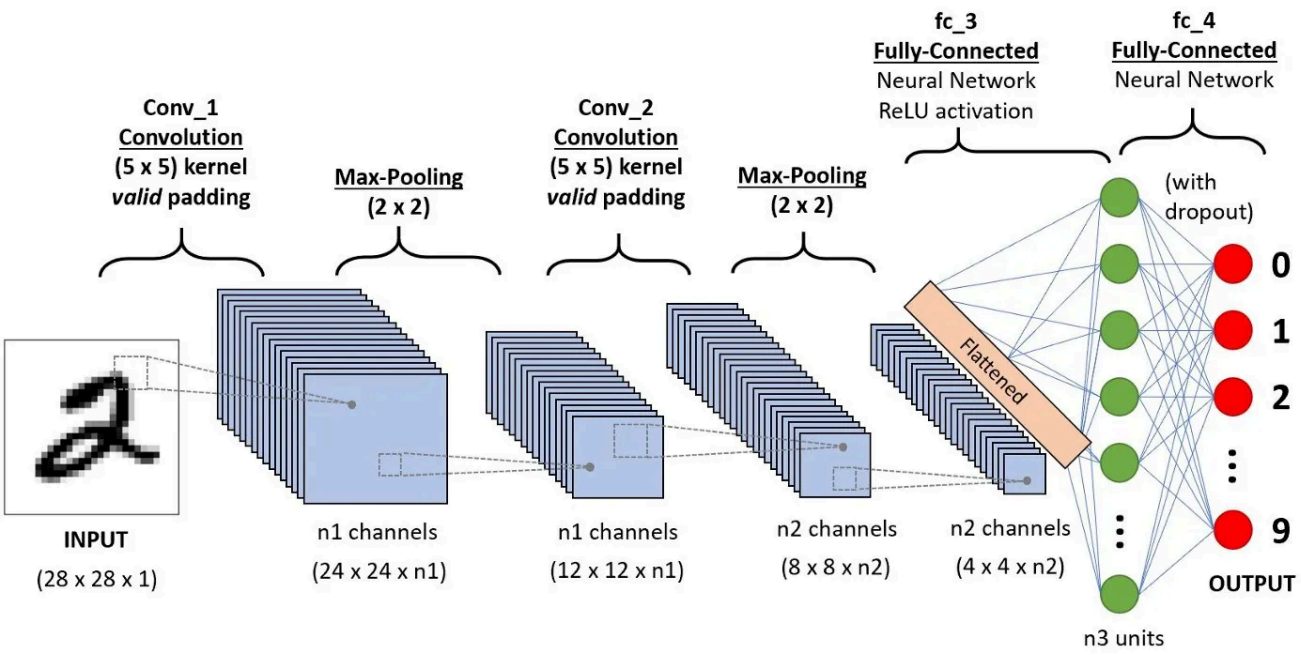
and produces the final classification scores.

Example: The 3072-length vector is connected to neurons for classification

7. **Output layer**

   The output layer converts final scores into probabilities using activation function like Sigmoid (binary classification) or softmax (multi-class classification)

   Example: For 10 classes, Softmax produces 10 probability values each representing the likelihood of a class.




```
class FashionMNISTModelV2(nn.Module):

def __init__(self, input_shape: int, hidden_units: int, output_shape: int):

super().__init__()

self.block_1 = nn.Sequential(

nn.Conv2d(in_channels=input_shape,

out_channels=hidden_units,

kernel_size=3, # how big is the square that's going over the image?

stride=1, # default

padding=1),# options = "valid" (no padding) or "same" (output has same shape as input) or int for specific number

nn.ReLU(),

nn.Conv2d(in_channels=hidden_units,
```

```
out_channels=hidden_units,

kernel_size=3,

stride=1,

padding=1),

nn.ReLU(),

nn.MaxPool2d(kernel_size=2,

stride=2) # default stride value is same as kernel_size

)

self.block_2 = nn.Sequential(

nn.Conv2d(hidden_units, hidden_units, 3, padding=1),

nn.ReLU(),

nn.Conv2d(hidden_units, hidden_units, 3, padding=1),

nn.ReLU(),

nn.MaxPool2d(2)

)

self.classifier = nn.Sequential(

nn.Flatten(),

# Where did this in_features shape come from?

# It's because each layer of our network compresses and changes the shape of our
input data.

nn.Linear(in_features=hidden_units*7*7,

out_features=output_shape)

)



def forward(self, x: torch.Tensor):
```

```
x = self.block_1(x)

# print(x.shape)

x = self.block_2(x)

# print(x.shape)

x = self.classifier(x)

# print(x.shape)

return x



torch.manual_seed(42)

model_2 = FashionMNISTModelV2(input_shape=1,

hidden_units=10,

output_shape=len(class_names)).to(device)

model_2
```